

Context API

Context Api solves Problem of Prop Drilling;

Context API allows you to **share state globally across a subtree** without manually passing props at every level.

3 Core Building Blocks

Step 1 — Create Context

```
import { createContext } from "react";  
  
const AuthContext = createContext();
```

Step 2 — Provide Value

```
<AuthContext.Provider value={{ user }}>  
  <App />  
</AuthContext.Provider>
```

Now every component inside can access user.

Step 3 Consume Context

Modern way (hooks):

```
import { useContext } from "react";  
  
const user = useContext(AuthContext);
```

Under The Hood (Important for Real Understanding)

When Provider's value changes:

☞ **ALL consuming components re-render**

Even if only one field changed.

Example:

```
<AuthContext.Provider value={{ user, theme }}>
```

If theme changes → all components using user also re-render.

That's a performance trap.

5 Correct Pattern (Professional Setup)

Create context file

```
// authContext.js
```

```
import { createContext, useState, useContext } from "react";
```

```
const AuthContext = createContext();
```

```
export function AuthProvider({ children }) {
```

```
  const [user, setUser] = useState(null);
```

```
  const login = (data) => setUser(data);
```

```
  const logout = () => setUser(null);
```

```
  return (
```

```
    <AuthContext.Provider value={{ user, login, logout }}>
```

```
      {children}
```

```
    </AuthContext.Provider>
```

```
  );
```

```
}
```

```
export function useAuth() {
```

```
  return useContext(AuthContext);
```

```
}
```

In index.js

```
<AuthProvider>  
<App />  
</AuthProvider>
```

// then anywhere

```
const { user, login } = useAuth();
```

Performance Optimization (Senior Level Knowledge)

Problem:

```
value={{ user, login, logout }}
```

This object is recreated every render.

So even if user didn't change → consumers re-render.

Fix:

```
import { useMemo } from "react";
```

```
const value = useMemo(() => ({  
  user,  
  login,  
  logout  
}), [user]);
```

Now re-renders only happen when user changes.



Step 0: Basic React App Structure

Imagine this structure:

```
src/  
└─ main.jsx (or index.js)
```

```
├── App.jsx
├── context/
│   └── AuthContext.jsx
└── components/
    └── Navbar.jsx
```

✅ Step 1: Create the Context File

Create:

`src/context/AuthContext.jsx`

Now write this:

```
import { createContext } from "react";
```

```
const AuthContext = createContext();
```

```
export default AuthContext;
```

What happened?

- `createContext()` creates a context object.
- This object gives us:
 - `AuthContext.Provider`
 - `AuthContext.Consumer`

Right now, it's empty. No logic yet.

✅ Step 2: Create a Provider Component

We now wrap our app in a provider.

Update `AuthContext.jsx`:

```
import { createContext, useState } from "react";
```

```
const AuthContext = createContext();
```

```
export function AuthProvider({ children }) {  
  const [user, setUser] = useState(null);
```

```
const login = (name) => {
  setUser({ name });
};

const logout = () => {
  setUser(null);
};

return (
  <AuthContext.Provider value={{ user, login, logout }}>
    {children}
  </AuthContext.Provider>
);
}

export default AuthContext;
```

What is {children}?

When you wrap your app:

```
<AuthProvider>
  <App />
</AuthProvider>
```

children becomes <App />.

This is how the provider wraps your app.

Step 3: Wrap Your Entire App

Open main.jsx (or index.js).

Change it to:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import { AuthProvider } from "../context/AuthContext";
```

```
ReactDOM.createRoot(document.getElementById("root")).render(  
  <AuthProvider>  
    <App />  
  </AuthProvider>  
);
```

Now the entire app has access to AuthContext.